

Introdução a Algoritmos

Aula 12 – Structs

Raoni F. S. Teixeira · 2s/2026

Objetivos desta aula

Ao final desta aula, você deverá ser capaz de:

1. Declarar structs com typedef e acessar campos com os operadores . e ->, escolhendo o correto conforme o contexto;
2. Explicar a diferença entre passar uma struct por valor e por ponteiro, e identificar quando cada abordagem é preferível;
3. Substituir vetores paralelos por um vetor de structs, eliminando o risco de dessincronização entre campos relacionados;
4. Declarar e acessar structs aninhadas, encadeando operadores de acesso para chegar a campos de sub-estruturas.

1 O problema dos vetores paralelos

Imagine um programa que gerencia uma turma de alunos. Cada aluno tem nome, matrícula e média final. Com o que sabemos, a solução natural é usar três vetores separados:

```
char    nomes[50][100];  
int     matriculas[50];  
float   medias[50];
```

Isso funciona, mas cria um problema sutil: os três vetores são independentes para o compilador, mas logicamente representam **a mesma entidade** — o aluno de índice *i*. Se precisarmos ordenar os alunos por média, teremos que mover elementos nos três vetores simultaneamente e garantir que nunca se dessincronizem. Se adicionarmos um novo atributo — telefone, endereço, data de nascimento — criamos um quarto vetor paralelo.

Vetores paralelos são uma fonte clássica de bugs de manutenção. O problema é que a linguagem não sabe que `nomes[i]`, `matriculas[i]` e `medias[i]` pertencem juntos. O programador sabe, mas o compilador não.

A solução é ensinar ao compilador que esses três dados formam uma unidade: o **registro** de um aluno. Em C, isso se faz com `struct`.

2 Declaração de struct

Definição — Struct

Uma struct (estrutura) é um tipo de dado composto que agrupa variáveis de tipos possivelmente diferentes sob um único nome. Cada variável dentro da struct é chamada de **campo** ou **membro**.

Na Aula 1, apresentamos “dados” como a peça de guardar e organizar informação. Vetor e matriz organizam vários valores do **mesmo** tipo; a struct dá um passo diferente: organiza valores de tipos **diferentes** que descrevem uma mesma entidade. É a mesma peça, agora heterogênea.

```
struct Aluno {  
    char nome[100];  
    int matricula;  
    float media;  
};
```

Essa declaração define um novo **tipo** — struct Aluno — mas não cria nenhuma variável ainda. Para criar variáveis desse tipo:

```
struct Aluno a1;  
struct Aluno turma[50];
```

Os campos são acessados com o operador . (ponto):

```
a1.matricula = 20240001;  
a1.media = 8.5;
```

Para strings dentro de struct, usamos strcpy — nunca =:

```
strcpy(a1.nome, "Maria Silva"); /* correto */  
a1.nome = "Maria Silva";      /* ERRADO – nao se atribui array */
```

3 typedef: simplificando a sintaxe

Escrever struct Aluno em todo lugar é verboso. O typedef cria um **apelido** para um tipo existente, permitindo omitir a palavra struct:

```
/* Forma 1: struct separado do typedef */  
struct Aluno {  
    char nome[100];  
    int matricula;  
    float media;  
};  
typedef struct Aluno Aluno;  
  
/* Forma 2: struct e typedef juntos (mais comum) */  
typedef struct {  
    char nome[100];  
    int matricula;  
    float media;  
} Aluno;
```

Com typedef, as declarações ficam mais limpas:

```
Aluno a1;          /* em vez de: struct Aluno a1; */  
Aluno turma[50];  /* em vez de: struct Aluno turma[50]; */
```

Recomendação: use sempre typedef. A forma sem typedef é necessária em situações específicas — structs que se referenciam a si mesmas (listas encadeadas) exigem o nome struct X antes do typedef estar completo. Para o uso geral, typedef torna o código mais legível sem nenhuma desvantagem.

4 Inicialização e cópia

Structs podem ser inicializadas com lista, assim como vetores:

```
Aluno a1 = {"Maria Silva", 20240001, 8.5};
```

Diferente de vetores, structs **podem ser atribuídas diretamente** com =:

```
Aluno a2 = a1; /* copia todos os campos de a1 para a2 */
```

Isso copia campo a campo — incluindo arrays internos como nome. A cópia é independente: modificar a2.nome não afeta a1.nome.

5 Structs e funções

5.1 Passagem por valor

Uma struct pode ser passada por valor — o que copia todos os seus campos:

```
void imprime_aluno(Aluno a) {  
    printf("Nome: %s\n", a.nome);  
    printf("Mat: %d\n", a.matricula);  
    printf("Med: %.1f\n", a.media);  
}
```

Para structs pequenas isso é aceitável. Para structs grandes — com arrays internos volumosos — a cópia tem custo significativo.

5.2 Passagem por ponteiro e o operador ->

Para evitar a cópia, ou quando a função precisa modificar a struct original, passamos um ponteiro:

```
void aplica_bonus(Aluno *a, float bonus) {  
    a->media += bonus;  
    if (a->media > 10.0) a->media = 10.0;  
}
```

O operador -> acessa um campo através de um ponteiro. É um atalho para (*a).media — desreferenciar o ponteiro e acessar o campo. As duas formas são equivalentes; -> é preferida por ser mais legível.

```
Aluno a1 = {"Joao", 20240002, 7.0};  
aplica_bonus(&a1, 0.5);  
/* a1.media agora e 7.5 */
```

Regra prática: use -> quando trabalhar com ponteiro para struct; use . quando trabalhar diretamente com a struct. O compilador aceitaria (*ptr).campo, mas ptr->campo é universalmente preferido.

6 Vetor de structs

A combinação mais poderosa: substituir vetores paralelos por um único vetor de structs. Os dados de cada aluno ficam juntos, e ordenar ou processar a turma torna-se muito mais seguro:

```
#define MAX_TURMA 50

typedef struct {
    char   nome[100];
    int    matricula;
    float  media;
} Aluno;

void le_turma(Aluno turma[], int n) {
    for (int i = 0; i < n; i++) {
        printf("Nome: ");
        scanf("%99s", turma[i].nome);
        printf("Matricula: ");
        scanf("%d", &turma[i].matricula);
        printf("Media: ");
        scanf("%f", &turma[i].media);
    }
}

void imprime_turma(Aluno turma[], int n) {
    printf("%-20s %10s %6s\n", "Nome", "Matricula", "Media");
    printf("%-20s %10s %6s\n", "----", "-----", "-----");
    for (int i = 0; i < n; i++) {
        printf("%-20s %10d %6.1f\n",
            turma[i].nome,
            turma[i].matricula,
            turma[i].media);
    }
}

/* Retorna o indice do aluno com maior media */
int melhor_aluno(Aluno turma[], int n) {
    int idx = 0;
    for (int i = 1; i < n; i++)
        if (turma[i].media > turma[idx].media)
            idx = i;
    return idx;
}
```

Observe `turma[i].nome`, `turma[i].matricula`: o índice `[i]` seleciona o aluno, o ponto `.` seleciona o campo. A notação é natural e não admite ambiguidade — o compilador sabe exatamente a qual aluno cada campo pertence.

7 Structs aninhadas

Um campo de uma struct pode ser ele mesmo uma struct — structs aninhadas permitem modelar entidades compostas com sub-entidades:

```
typedef struct {
    int dia;
    int mes;
    int ano;
} Data;

typedef struct {
    char nome[100];
    int matricula;
    float media;
    Data nascimento; /* campo do tipo Data */
} Aluno;
```

O acesso encadeia operadores .:

```
Aluno a;
a.nascimento.dia = 15;
a.nascimento.mes = 3;
a.nascimento.ano = 2005;
printf("Nascido em: %02d/%02d/%d\n",
       a.nascimento.dia,
       a.nascimento.mes,
       a.nascimento.ano);
```

E por ponteiro:

```
Aluno *p = &a;
printf("%d\n", p->nascimento.ano); /* -> para o ponteiro, . para o campo interno */
```

Exemplo — Calcular idade a partir da data de nascimento

```
int idade(Data nasc, Data hoje) {
    int anos = hoje.ano - nasc.ano;
    /* ajusta se ainda nao fez aniversario este ano */
    if (hoje.mes < nasc.mes ||
        (hoje.mes == nasc.mes && hoje.dia < nasc.dia))
        anos--;
    return anos;
}

/* Uso */
Data hoje = {15, 3, 2025};
printf("Idade: %d anos\n", idade(a.nascimento, hoje));
```

8 Exemplo integrador: cadastro de turma com structs aninhadas

O programa a seguir usa Aluno com Data aninhada, vetor de structs, passagem por ponteiro e as funções auxiliares desenvolvidas ao longo da aula:

```
#include <stdio.h>
#include <string.h>
#define MAX 50

typedef struct { int dia, mes, ano; } Data;

typedef struct {
```

```

    char nome[100];
    int matricula;
    float media;
    Data nascimento;
} Aluno;

void le_aluno(Aluno *a) {
    printf(" Nome: "); scanf("%99s", a->nome);
    printf(" Matricula: "); scanf("%d", &a->matricula);
    printf(" Media: "); scanf("%f", &a->media);
    printf(" Nascimento (dd mm aaaa): ");
    scanf("%d %d %d", &a->nascimento.dia,
                &a->nascimento.mes,
                &a->nascimento.ano);
}

float media_turma(Aluno turma[], int n) {
    float soma = 0;
    for (int i = 0; i < n; i++) soma += turma[i].media;
    return soma / n;
}

void aprovados(Aluno turma[], int n, float minimo) {
    printf("Aprovados (media >= %.1f):\n", minimo);
    for (int i = 0; i < n; i++)
        if (turma[i].media >= minimo)
            printf(" %s (%.1f)\n", turma[i].nome, turma[i].media);
}

int main() {
    Aluno turma[MAX];
    int n;
    printf("Numero de alunos: "); scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("Aluno %d:\n", i + 1);
        le_aluno(&turma[i]);
    }
    printf("\nMedia da turma: %.2f\n", media_turma(turma, n));
    aprovados(turma, n, 5.0);
    return 0;
}

```

9 Exercícios

1. Declare uma struct Ponto com campos float x e float y. Escreva funções float distancia(Ponto a, Ponto b) e Ponto ponto_medio(Ponto a, Ponto b). Note que ponto_medio **retorna** uma struct — isso é válido em C e copia todos os campos.
2. Declare uma struct Fracao com campos int num e int den. Escreva funções para as quatro operações aritméticas entre frações, simplificando o resultado usando o MDC (reutilize a função do exercício da Aula 7).
3. Adicione ao programa da turma uma função void ordena_por_media(Aluno turma[], int n) que ordena o vetor em ordem decrescente de média usando o algoritmo de ordenação por seleção: encontre o maior elemento, troque-o com o primeiro; depois o segundo maior com o segundo; e

assim por diante. Observe que trocar dois alunos é uma única atribuição de struct — não precisa trocar campo a campo.

4. (*Desafio*) Declare structs Endereco e Contato (com nome, telefone e endereço aninhado). Escreva um programa de agenda que armazena até 100 contatos e permite: adicionar, buscar por nome (retorna ponteiro para o contato ou NULL se não encontrado), e listar todos em ordem alfabética.